

Práctica 1: Repaso de Consultas SQL y Uso de Servidores Vinculados

Materia: Base de Datos Distribuidas (Grupo 3TM2)

Consulta 1: Top 10 productos más vendidos (Enfoque en Volumen y Precio)

Objetivo de la consulta: Encontrar los 10 productos más vendidos durante 2014, obteniendo el nombre del producto, la cantidad total vendida y el cliente que lo compró. Posteriormente, desarrollé una variante para analizar el aspecto económico, agregando el precio unitario promedio y filtrando únicamente los productos "caros" (con un precio de lista mayor a 1000).

Mi análisis y conclusión: Ambas versiones las resolví utilizando la misma estrategia estructural: Expresiones de Tabla Comunes (CTEs). Primero creé un conjunto base con las ventas del 2014, luego agrupé los datos para ver cuánto se vendió por producto, calculé las compras totales por cliente y finalmente extraje el top 10. La principal diferencia es que la primera consulta se enfoca puramente en el volumen de ventas, mientras que la segunda tiene un enfoque más económico al incluir información de precios y filtrar por un valor mínimo.

SQL

-- Variante final con análisis de precio (Corregida para 2014):

WITH ProductSales2014 AS (

SELECT

pr.ProductID, pr.Name AS ProductName, pr.ListPrice,
sod.SalesOrderID, soh.OrderDate, c.CustomerID,
CONCAT(p.FirstName, ' ', p.LastName) AS CustomerName,
sod.OrderQty, sod.UnitPrice

FROM Sales.SalesOrderHeader soh

JOIN Sales.SalesOrderDetail sod ON sod.SalesOrderID = soh.SalesOrderID

JOIN Production.Product pr ON pr.ProductID = sod.ProductID

JOIN Sales.Customer c ON c.CustomerID = soh.CustomerID

JOIN Person.Person p ON p.BusinessEntityID = c.PersonID

WHERE soh.OrderDate >= '20140101' AND soh.OrderDate < '20150101')

),

Agg AS (

SELECT

ProductID, ProductName, ListPrice,

SUM(OrderQty) AS AmountOfSales, avg(UnitPrice) as AvgUnitPrice

FROM ProductSales2014

GROUP BY ProductID, ProductName, ListPrice

),

SalesByCustomer as (

select distinct (CustomerName), sum(OrderQty) as TotalBought

from ProductSales2014

group by CustomerName

),

T10Products as (

select top 10 ProductID

from Agg

where ListPrice >= 1000

order by AmountOfSales desc

)

select

a.ProductName, a.AmountOfSales, sbc.CustomerName,

sbc.TotalBought, a.AvgUnitPrice, a.ListPrice

from Agg a

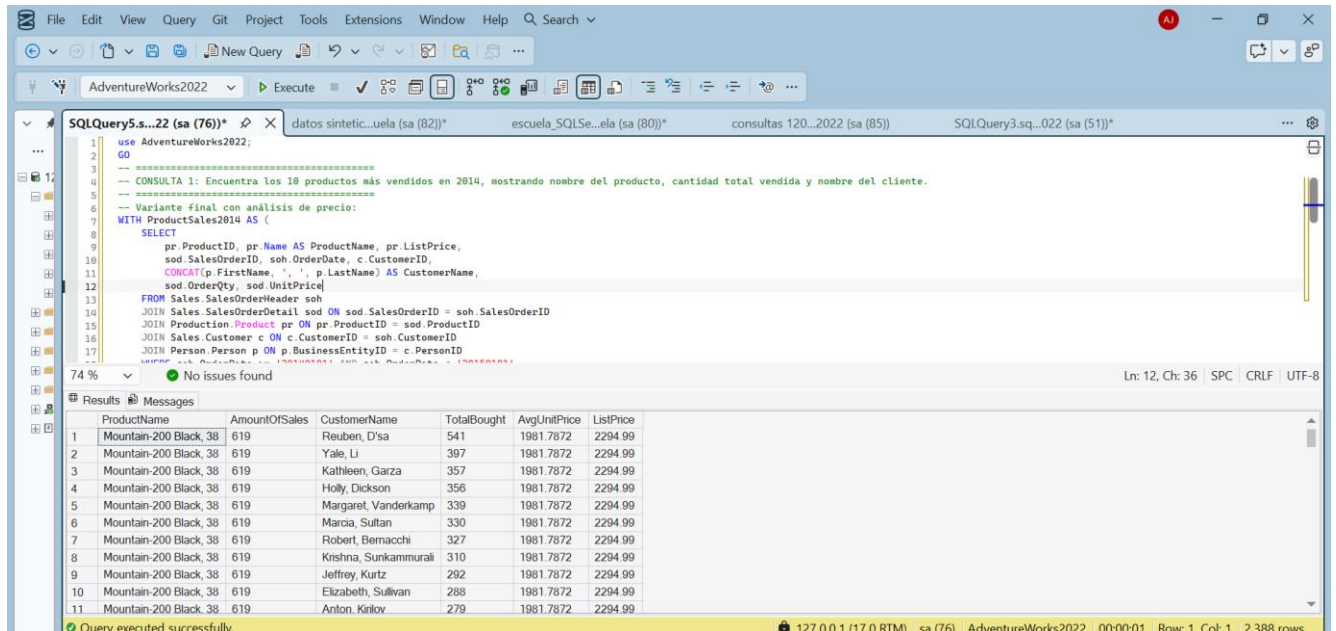
join ProductSales2014 ps on a.ProductID = ps.ProductID

join SalesByCustomer sbc on ps.CustomerName = sbc.CustomerName

where a.ProductID in (select * from T10Products)

group by a.ProductName, a.AmountOfSales, sbc.CustomerName, sbc.TotalBought,
a.AvgUnitPrice, a.ListPrice

order by a.AmountOfSales desc, sbc.TotalBought desc;



The screenshot shows a SQL query execution in SQL Server Enterprise Manager. The query is a complex SQL statement using CTEs to find the top 10 products by sales in 2014. The results table shows 11 rows of product data.

	ProductName	AmountOfSales	CustomerName	TotalBought	AvgUnitPrice	ListPrice
1	Mountain-200 Black, 38	619	Reuben, D'sa	541	1981.7872	2294.99
2	Mountain-200 Black, 38	619	Yale, Li	397	1981.7872	2294.99
3	Mountain-200 Black, 38	619	Kathleen, Garza	357	1981.7872	2294.99
4	Mountain-200 Black, 38	619	Holly, Dickson	356	1981.7872	2294.99
5	Mountain-200 Black, 38	619	Margaret, Vanderkamp	339	1981.7872	2294.99
6	Mountain-200 Black, 38	619	Marcia, Sultan	330	1981.7872	2294.99
7	Mountain-200 Black, 38	619	Robert, Bernacchi	327	1981.7872	2294.99
8	Mountain-200 Black, 38	619	Krishna, Sunkammurall	310	1981.7872	2294.99
9	Mountain-200 Black, 38	619	Jeffrey, Kurtz	292	1981.7872	2294.99
10	Mountain-200 Black, 38	619	Elizabeth, Sullivan	288	1981.7872	2294.99
11	Mountain-200 Black, 38	619	Anton, Kirov	279	1981.7872	2294.99

Consulta 2: Empleados con ventas superiores al promedio regional

Objetivo de la consulta: Listar a los empleados que lograron vender más que el promedio de ventas por empleado en el territorio 'Northwest' (TerritoryID = 1).

Mi análisis y conclusión: Inicialmente resolví este problema utilizando subconsultas tradicionales. Aunque es una buena forma de empezar a construir la lógica paso a paso, el código se vuelve muy largo y es fácil perderse entre tantas líneas. Por ello, refactoricé el código usando CTEs. Al hacerlo de esta forma, me di cuenta de que puedes manejar las tablas temporales casi como si fueran variables en programación, moviéndolas y manipulándolas más fácilmente. Esto resulta en un ahorro de recursos significativo si se trata de una consulta que se ejecutará constantemente.

SQL

-- Solución optimizada usando CTEs:

WITH SumaXEmpleado AS (

 Select sum(TotalDue) as SumaPorEmpleado, Person.BusinessEntityID as IDVendedor

 from Sales.SalesOrderHeader

 join Person.Person on Sales.SalesOrderHeader.SalesPersonID = Person.BusinessEntityID

 where SalesOrderHeader.TerritoryID = 1 and Person.PersonType = 'SP'

 group by Person.BusinessEntityID

),

PromedioEmpleado as (

 select AVG(SumaPorEmpleado) as PromedioXEmpleado

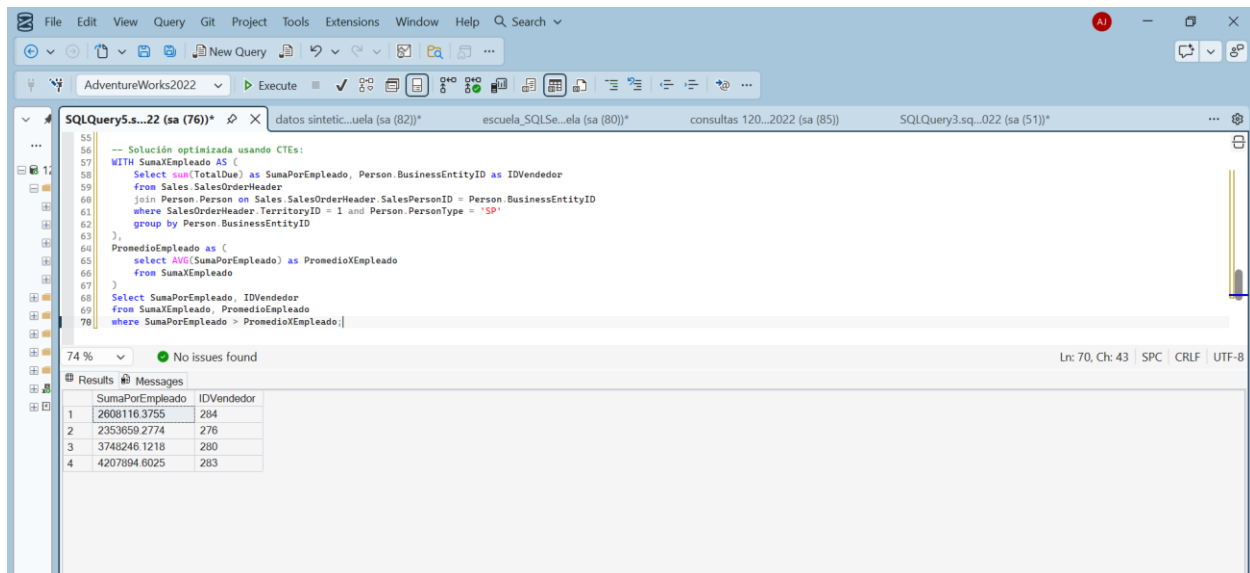
 from SumaXEmpleado

)

Select SumaPorEmpleado, IDVendedor

from SumaXEmpleado, PromedioEmpleado

where SumaPorEmpleado > PromedioXEmpleado;



The screenshot shows a SQL Server Enterprise Manager interface. The top pane displays a query window with the following SQL code:

```
-- Solución optimizada usando CTEs:
WITH SumaXEmpleado AS (
    Select sum(TotalDue) as SumaPorEmpleado, Person.BusinessEntityID as IDVendedor
    from Sales.SalesOrderHeader
    join Person.Person on Sales.SalesOrderHeader.SalesPersonID = Person.BusinessEntityID
    where SalesOrderHeader.TerritoryID = 1 and Person.PersonType = 'SP'
    group by Person.BusinessEntityID
),
PromedioEmpleado as (
    select AVG(SumaPorEmpleado) as PromedioXEmpleado
    from SumaXEmpleado
)
Select SumaPorEmpleado, IDVendedor
from SumaXEmpleado, PromedioEmpleado
where SumaPorEmpleado > PromedioXEmpleado;
```

The bottom pane shows the results of the query, displaying a table with two columns: SumaPorEmpleado and IDVendedor. The results are as follows:

SumaPorEmpleado	IDVendedor
2608116.3755	284
2353659.2774	276
3748246.1218	280
4207894.6025	283

Consulta 3: Ventas totales por territorio y año (Filtros y Desviación Estándar)

Objetivo de la consulta: Calcular las ventas agrupadas por territorio y año, mostrando únicamente aquellos casos que superen las 5 órdenes y que tengan ventas mayores a 1 millón. Además, implementé una variante para agregar el cálculo de la desviación estándar.

Mi análisis y conclusión: Para la primera versión, realicé varios JOIN relacionando los territorios con las cabeceras y detalles de las órdenes, usando SUM(LineTotal) para obtener el importe total y filtrando con HAVING. Para lograr integrar la desviación estándar en la segunda variante, dividí el proceso en dos etapas mediante una CTE: primero agrupé y calculé los totales por cada orden individual, y después, en la consulta principal, volví a agrupar por año y territorio usando la función STDEV para sacar la desviación, filtrando las órdenes mayores o iguales a 5.

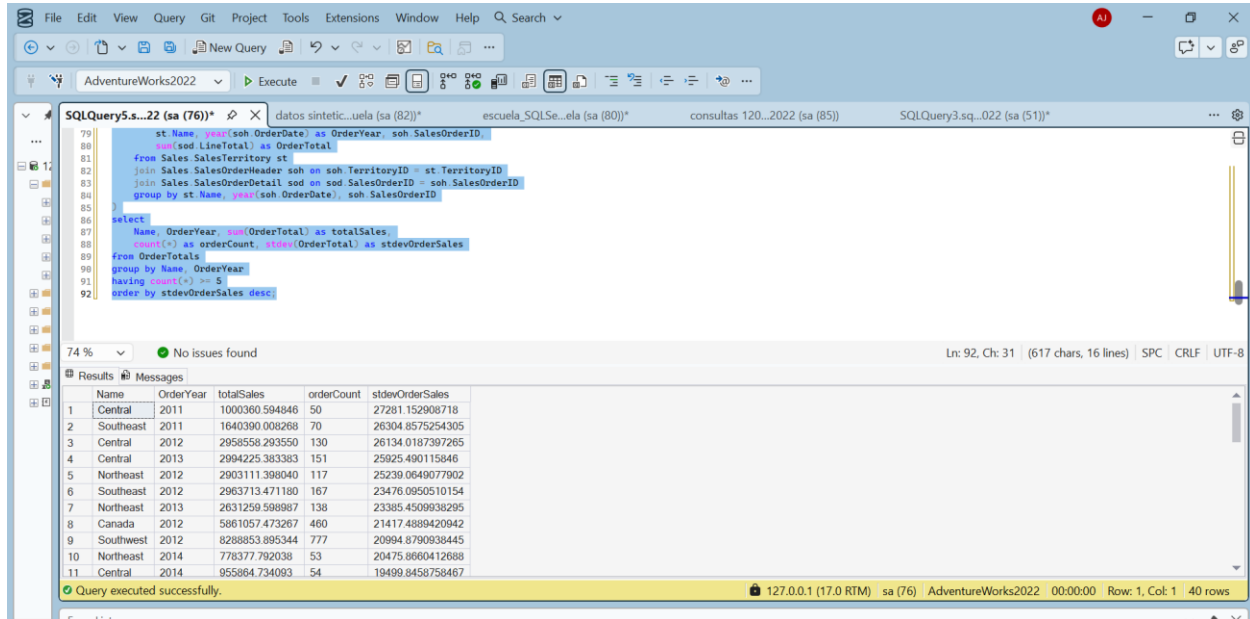
SQL

-- Variante con cálculo de Desviación Estándar:

```
with OrderTotals as (
    select
        st.Name, year(soh.OrderDate) as OrderYear, soh.SalesOrderID,
        sum(sod.LineTotal) as OrderTotal
    from Sales.SalesTerritory st
    join Sales.SalesOrderHeader soh on soh.TerritoryID = st.TerritoryID
    join Sales.SalesOrderDetail sod on sod.SalesOrderID = soh.SalesOrderID
    group by st.Name, year(soh.OrderDate), soh.SalesOrderID
)
select
    Name, OrderYear, sum(OrderTotal) as totalSales,
    count(*) as orderCount, stdev(OrderTotal) as stdevOrderSales
from OrderTotals
group by Name, OrderYear
```

having count(*) >= 5

order by stdevOrderSales desc;



The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays a SQL query that uses Common Table Expressions (CTEs) to calculate total sales and standard deviation of order sales for each territory. The query is as follows:

```
79  with st as (select soh.OrderDate as OrderYear, soh.SalesOrderID,
80  sum(sod.LineTotal) as OrderTotal
81  from Sales.SalesTerritory st
82  join Sales.SalesOrderHeader soh on soh.TerritoryID = st.TerritoryID
83  join Sales.SalesOrderDetail sod on sod.SalesOrderID = soh.SalesOrderID
84  group by st.Name, soh.OrderDate, soh.SalesOrderID
85  ), ot as (
86  select
87  Name, OrderYear, sum(OrderTotal) as totalSales,
88  count(*) as orderCount, stdev(OrderTotal) as stdevOrderSales
89  from OrderTotals
90  group by Name, OrderYear
91  having count(*) >= 5
92  order by stdevOrderSales desc;
```

The results pane shows the following data:

Name	OrderYear	totalSales	orderCount	stdevOrderSales
1 Central	2011	1000360.594846	50	27281.152908718
2 Southeast	2011	1640390.008268	70	26304.8575254305
3 Central	2012	2958558.293550	130	26134.0187397265
4 Central	2013	2994225.383383	151	25925.490115846
5 Northeast	2012	2903111.398040	117	25230.0649077902
6 Southeast	2012	2963713.471180	167	23476.0950510154
7 Northeast	2013	2631259.598987	138	23385.4509938295
8 Canada	2012	5861057.473267	460	21417.4889420942
9 Southwest	2012	8288853.895344	777	20994.8790938445
10 Northeast	2014	778377.792038	53	20475.8660412688
11 Central	2014	955864.734093	54	19499.8458758467

The status bar at the bottom indicates that the query was executed successfully and returned 40 rows.

Consulta 4: División Relacional (Vendedores que manejan categorías completas)

Objetivo de la consulta: Encontrar a los vendedores que han vendido TODOS los productos existentes dentro de una categoría específica (por ejemplo, la categoría "Bikes" con ID=1 o "Clothing" con ID=3).

Mi análisis y conclusión: Esta fue una de las consultas más complejas del ejercicio. Para que fuera más fácil de entender y visualizar, apliqué el concepto de División Relacional visto en clase, separando el problema en partes más sencillas utilizando CTEs. Primero, hice una consulta para contar el total de artículos que existen en la categoría deseada. Segundo, calculé el número de productos distintos que ha vendido cada vendedor. Finalmente, uní ambas tablas filtrando únicamente a aquellos vendedores cuyo número de productos vendidos fuera exactamente igual al total de artículos de la categoría. Para evaluar la categoría "Clothing", simplemente modifiqué el código cambiando el ID a 3.

SQL

-- Solución para la categoría "Bikes" (ID=1):

WITH NoProdBikes AS (

select COUNT(Productos.ProductID) as TotalArticulosBikes

from Production.ProductSubcategory as SubCat

join Production.ProductCategory as Cat on SubCat.ProductCategoryID =
Cat.ProductCategoryID

JOIN Production.Product as Productos on Productos.ProductSubcategoryID =
SubCat.ProductSubcategoryID

where Cat.ProductCategoryID = 1

),

ProdDistintXVende AS (

SELECT CabezaVenta.SalesPersonID, COUNT(DISTINCT Productos.ProductID) as
ProdDistintos

from Sales.SalesOrderHeader as CabezaVenta

join Person.Person as Empleados on CabezaVenta.SalesPersonID =
Empleados.BusinessEntityID

join Sales.SalesOrderDetail as DetalleVenta on CabezaVenta.SalesOrderID =
DetalleVenta.SalesOrderID

join Production.Product as Productos on Productos.ProductID = DetalleVenta.ProductID

join Production.ProductSubcategory as SubCat on Productos.ProductSubcategoryID =
SubCat.ProductSubcategoryID

join Production.ProductCategory as CategoriP on CategoriP.ProductCategoryID =
SubCat.ProductCategoryID

where CategoriP.ProductCategoryID = 1

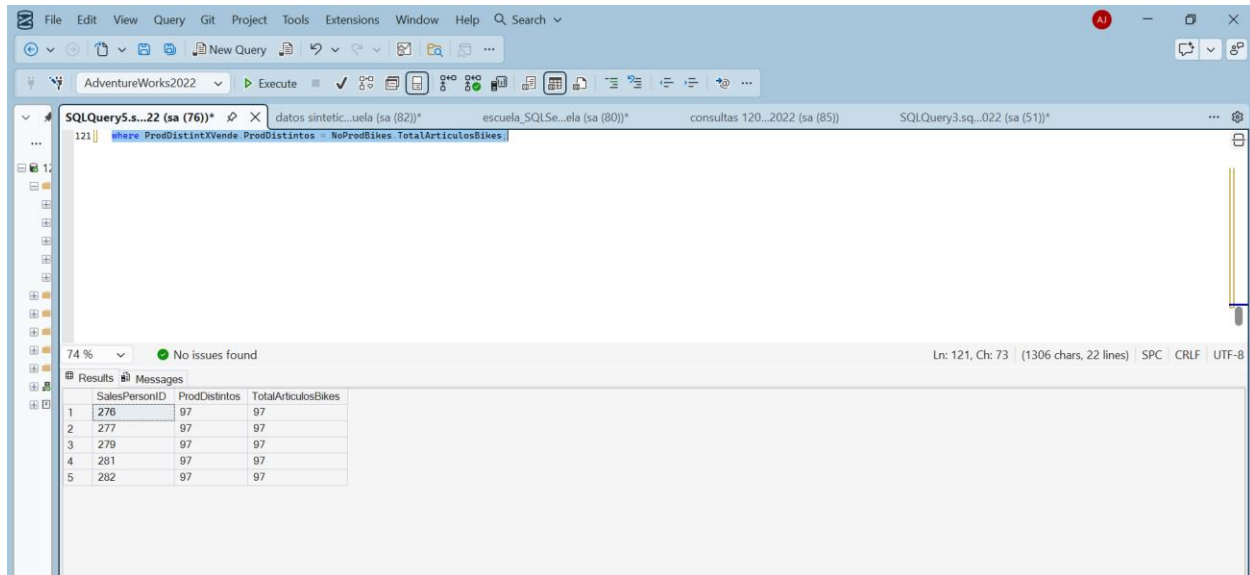
group by CabezaVenta.SalesPersonID

)

Select *

from ProdDistintXVende, NoProdBikes

where ProdDistintXVende.ProdDistintos = NoProdBikes.TotalArticulosBikes;



Consulta 5: Entornos Distribuidos (Producto más vendido por categoría)

Objetivo de la consulta: Determinar cuál es el producto número uno en ventas dentro de cada categoría, simulando un entorno de bases de datos distribuidas. El escenario asume que el esquema Sales vive en una instancia/servidor A y el esquema Production en un servidor B.

Mi análisis y conclusión: Para resolver este escenario distribuido, utilicé prefijos de servidores vinculados (SV_P1 para ventas y SV_P2 para producción) en las tablas al hacer los JOIN. Después, para poder extraer únicamente el producto top de cada categoría, utilicé la función de ventana ROW_NUMBER() OVER (PARTITION BY ProductCategoryID ORDER BY TotalSold DESC). Esto me generó un ranking de ventas agrupado por categoría, lo que me permitió finalmente hacer un filtrado sencillo donde la posición del ranking (rn) fuera igual a 1.

SQL

-- Solución con simulación de Servidores Vinculados:

WITH ProductSales AS (

SELECT

```

    pc.ProductCategoryID, pc.Name AS CategoryName,
    p.ProductID, p.Name AS ProductName,
    SUM(sod.OrderQty) AS TotalSold
FROM SV_P1.Sales.SalesOrderDetail sod
JOIN SV_P2.Production.Product p ON p.ProductID = sod.ProductID
JOIN Production.ProductSubcategory ps ON ps.ProductSubcategoryID =
p.ProductSubcategoryID
JOIN Production.ProductCategory pc ON pc.ProductCategoryID = ps.ProductCategoryID
GROUP BY pc.ProductCategoryID, pc.Name, p.ProductID, p.Name
),
RankedProducts AS (
    SELECT *,
    ROW_NUMBER() OVER (
        PARTITION BY ProductCategoryID
        ORDER BY TotalSold DESC
    ) AS rn
FROM ProductSales
)
SELECT CategoryName, ProductName, TotalSold
FROM RankedProducts
WHERE rn = 1
ORDER BY CategoryName;

```

File Edit View Query Git Project Tools Extensions Window Help Search

AdventureWorks2022 Execute

SQLQuery5.s...22 (sa (76))* datos sintetic...uela (sa (82))* escuela_SQLSe...ela (sa (80))* consultas 120...2022 (sa (85)) SQLQuery3.sq...022 (sa (51))*

```
122 --
123 --
124 -- CONSULTA 5: Determinar el producto más vendido de cada categoría de producto, considerando el escenario de que el esquema SALES se encuentra en una instancia (servidor) A y el esquema PRODUCTION en ot:
125 --
126 -- Solución con simulación de Servidores Vinculados:
127 WITH ProductSales AS (
128     SELECT
129         pc.ProductCategoryID,
130         pc.Name AS CategoryName,
131         p.ProductID,
132         p.Name AS ProductName,
133         SUM(sod.OrderQty) AS TotalSold
134     FROM Sales.SalesOrderDetail sod -- Se quitó SV_P1.
135     JOIN Production.Product p
136         ON p.ProductID = sod.ProductID -- Se quitó SV_P2.
137     JOIN Production.ProductSubcategory ps
138         ON ps.ProductSubcategoryID = p.ProductSubcategoryID
139 )
140 SELECT
141     CategoryName,
142     ProductName,
143     TotalSold
144 FROM ProductSales
145 ORDER BY TotalSold DESC
```

74 % No issues found Ln: 127, Ch: 1 (948 chars, 35 lines) SPC CRLF UTF-8

Results Messages

	CategoryName	ProductName	TotalSold
1	Accessories	Water Bottle - 30 oz.	6815
2	Bikes	Mountain-200 Black, 38	2977
3	Clothing	AWC Logo Cap	8311
4	Components	LL Road Frame - Black, 52	1581